Mirror Checker Assignment Report

IoT Standards & Protocols

SETU Waterford

Lecturer: Frank Walsh

Student: Darragh Drohan (ID: 20105993)

Submission Date: 04/05/2026

Table of Contents

Introduction

For this assignment we were asked to design and develop a project that utilises sensors connected to a Raspberry Pi which processes the sensor data and acts as a gateway to communicate with an application on a remote server, utilising suitable IoT protocols and standards where applicable. For this project I used a **Pi Camera Module 2** as my sensor connected to a **Raspberry Pi Model 4** which sends messages, using the Message Queuing Telemetry Transport (**MQTT**) protocol, to a remote server where the data is stored in a **MongoDB** collection and subsequently displayed in a web application using **Flask**.

Project Idea

The idea for my project was an IoT device that would use computer vision to monitor the head movements of a learner driver to evaluate if they are making the correct, pronounced head movements to check their car mirrors before maneuvers such as slowing down, changing lanes or turning corners. During the preliminary stages of the project, it was envisioned that other sensors would be used to determine when the car was performing a maneuver, however this idea would prove to be beyond the scope of the project and events are instead registered through a command line interface (CLI).

Project Implementation

The programming language used for this project was **Python 3.11** due to Python's speed of development and the availability of all the libraries required for this project (Paho MQTT, Picamera2, MediaPipe). The project is divided into two programs, **main.py** which runs on the Raspberry Pi and **server.py** which runs on a remote machine.

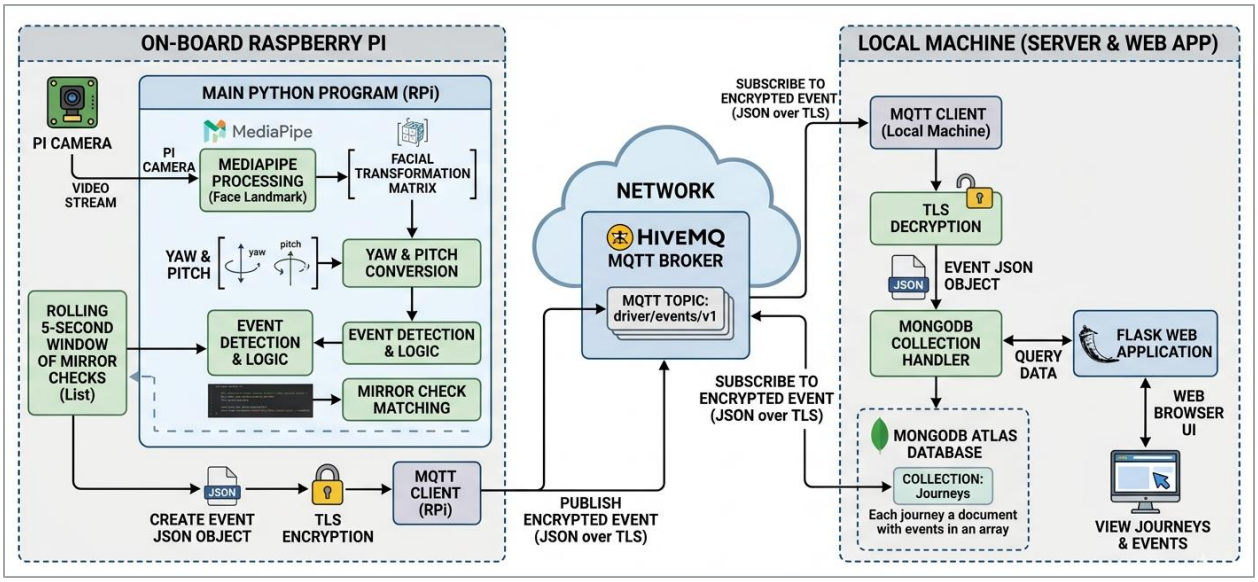

Figure 1: System Architecture Diagram. Created by Google Gemini [1].

main.py

Sensing and Processing

This program is a menu-based application with two options, calibration and starting a new car journey. Calibration allows a user to set the angle of their head when looking at four positions: looking straight ahead, at the rear-view mirror, and at the left or right car door mirrors. Frames are constantly captured by the Pi Camera and every n^{th} frame is analysed for head position (default: 3 frames). Analysing every n^{th} frame reduces the demand placed on the pi's computing capabilities. These frames are analysed by a pre-trained machine learning model, Google's **MediaPipe Face Landmarker** [2]. Part of the returned data from the MediaPipe model is a *facial transformation matrix*, which can be used to calculate the yaw, pitch, and roll of the head. Only the yaw and pitch are needed to calculate the driver's head position.

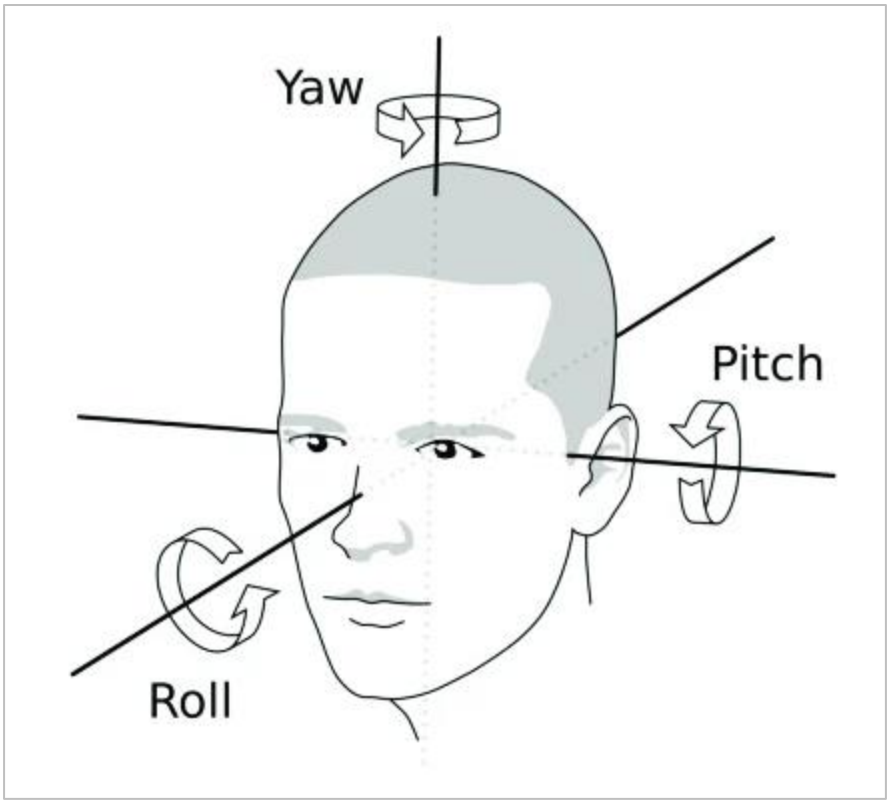

Figure 2: The yaw, pitch and roll of the human head. Image taken from [3].

The driver's head position is then evaluated for distance from each of the calibrated positions, matching the closest position. It is then determined whether the current head position is within a threshold number of degrees from the closest position (default: 10 degrees). In the case that it is within the threshold, the name and a timestamp of the mirror check are added to a double-ended queue (deque).

```python
def validate_check(yaw, pitch):
    closest_name = None
    closest_distance = float('inf')

    for name, centre in centres.items():
        dist = distance_from(yaw, pitch, centre['yaw'], centre['pitch'])

        if dist < closest_distance:
            closest_distance = dist
            closest_name = name

    if closest_distance < CENTRE_TOLERANCE_DEGREES and closest_name != last_checked:
        return closest_name
    return None

def distance_from(yaw1, pitch1, yaw2, pitch2):
    yaw_dist = (yaw1 - yaw2)
    pitch_dist = (pitch1 - pitch2)

    return np.hypot(yaw_dist, pitch_dist) # sqrt( (yaw * yaw) + (pitch * pitch) )
```

Figure 3: The functions used to determine the *Euclidean distance* between the yaw and pitch of the current head position and the yaw and pitch of the head calibrated at each mirror.

When an event is registered with the system, the deque is checked. Before checking occurs the deque is pruned to remove any mirror checks that occurred over *t* seconds ago (default: 5 seconds). Then the content of the deque is compared against a dictionary containing a list of the required mirror checks for each maneuver. If all the required mirror checks are present in the deque, then the driver's mirror checking for the maneuver was correct.

Communicating via MQTT

For each event, regardless of the outcome, a message, formatted into **JavaScript Object Notation (JSON)** format, containing details of the result of the maneuver is sent using the MQTT protocol to a topic on a public MQTT broker. The message is given a timestamp, encrypted using **symmetric encryption**, and sent using the **Transport Layer Security (TLS)** protocol; this ensures that even if someone were to subscribe to the topic, the drivers' data would be encrypted. Messages are sent using a Quality of Service (QoS) level of one, ensuring that each message will be sent *at least* once. Message IDs are tracked in a list and only removed from this list when an acknowledgement of receipt, a *PUBACK,* is received from the MQTT broker. This list allows for pending messages to be delivered before exiting the program, with a maximum of *t* seconds given after a user exits to send unacknowledged messages (default: 20).

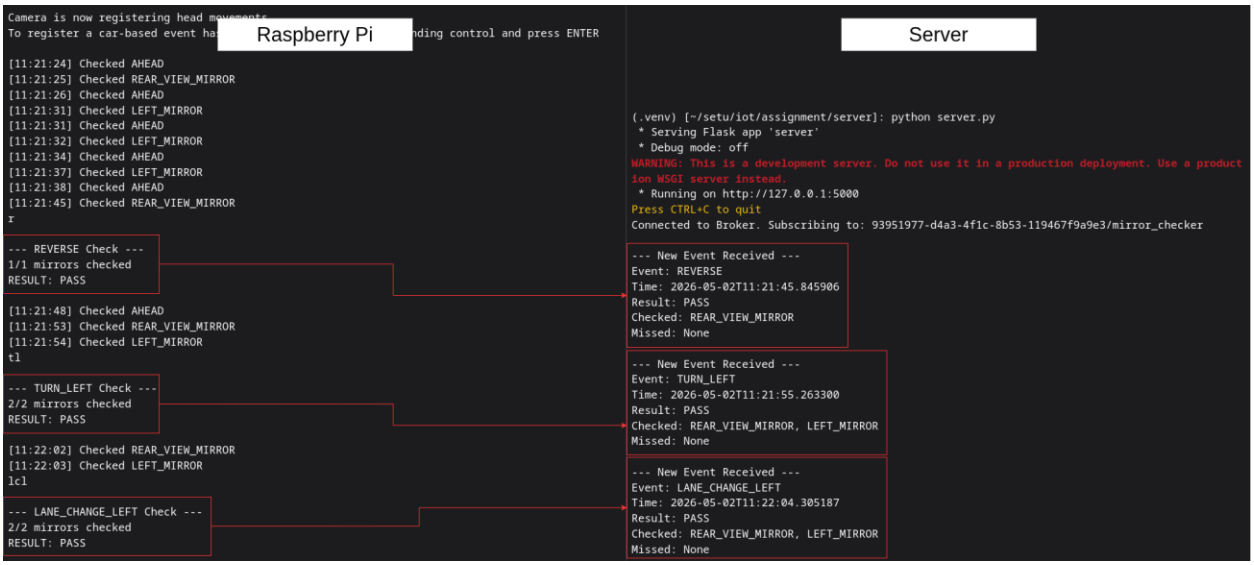

Figure 4: Messages being sent via the MQTT protocol

server.py

Persisting and Displaying the Data

The Python program running on the server has two functions. Firstly, in a separate thread from the main thread it subscribes to the MQTT topic to receive updates from the Raspberry Pi. These messages are then stored in a MongoDB collection with a new document for each unique journey ID, which is set on the Raspberry Pi. New messages received are appended to an array in the relevant journey's document. The server maintains a set of message IDs it has been sent, ignoring any duplicate messages already received.

Secondly, it runs a Flask web application which displays a list of all journeys with the most recent journey at the top. Each journey has a separate page which contains a safety score, number of passed maneuvers divided by total number of maneuvers, and an ordered list of the names of the maneuvers performed, and the mirrors checked or missed for each maneuver. Both the list of journeys and the list of maneuvers for each journey are updated in real time using **Flask-SocketIO**.

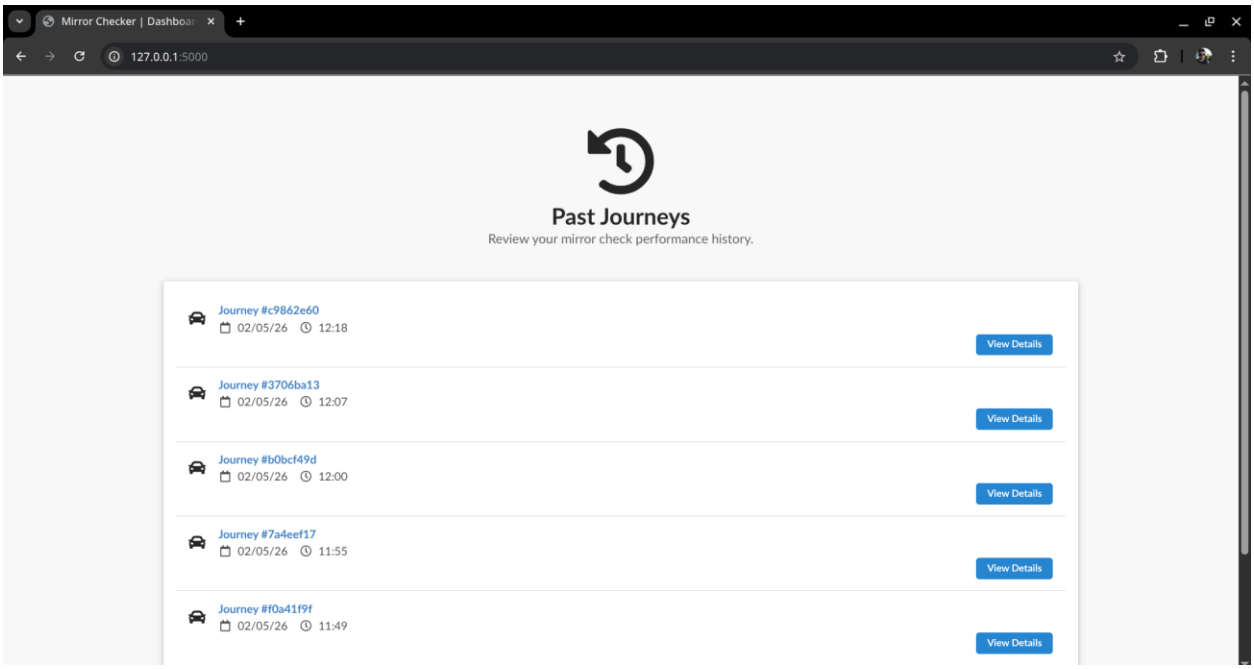

Figure 5: The index page displaying the most recent journeys

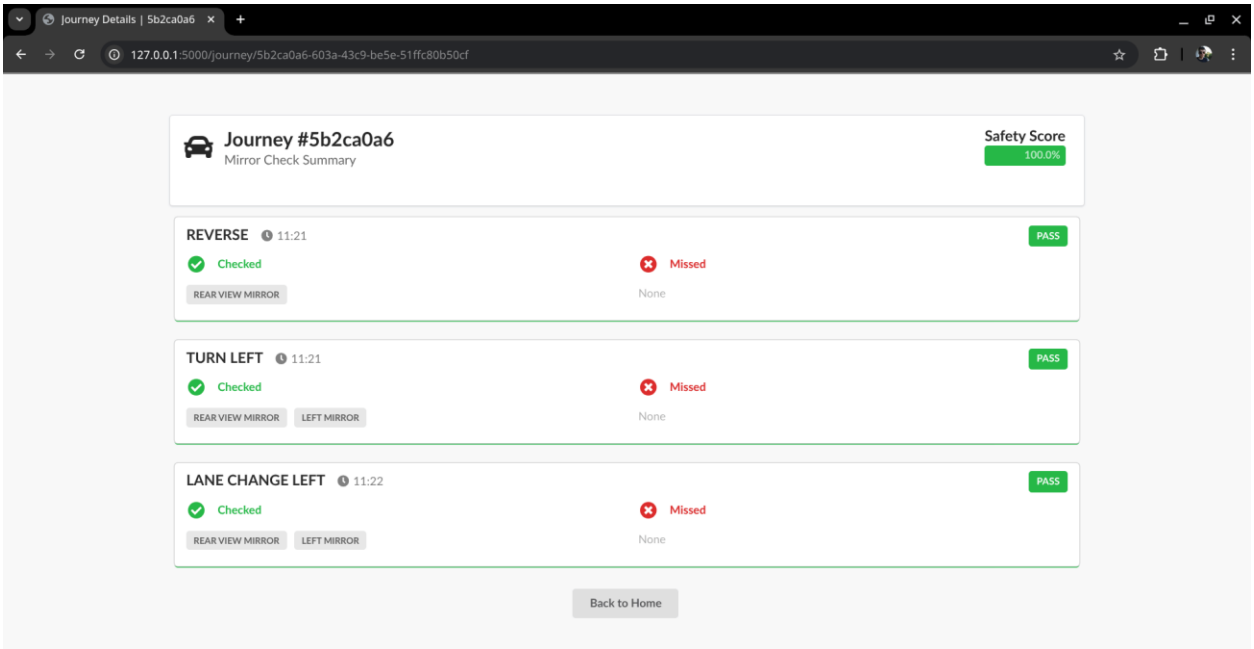

Figure 6: Details of mirror checks during a journey

Generative AI Disclosure

Generative AI (Google Gemini) was used in several places in this project, detailed in the following list:

- The system architecture diagram in this report [1].
- The HTML templates used for the flask app, available in server/templates/ [4].
- The JavaScript which uses Flask-SocketIO for real-time updates for the web app [5].
- The function to convert the facial transformation matrix to yaw and pitch in main.py [6].

The conversations are available in the references section of this report. Where relevant, the usage of AI has also been commented above the AI generated code.

References

[1] Image generated by Google Gemini. Conversation is available here: https://gemini.google.com/share/600ca7dd4091

[2] Google. (n.d.). *Face landmarker guide*. Google AI Edge. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

[3] Jantunen, T., Mesch, J., Puupponen, A., Laaksonen, J. (2016) *On the rhythm of head movements in Finnish and Swedish Sign Language sentences*. Proc. Speech Prosody 2016, 850-853, doi: 10.21437/SpeechProsody.2016-174

[4] HTML generated by Google Gemini. Conversation is available here: https://gemini.google.com/share/d386ccbd61f3

[5] JavaScript generated by Google Gemini. Conversation is available here: https://gemini.google.com/share/35d493197829

[6] Python generated by Google Gemini. Conversation is available here: https://gemini.google.com/share/33f81795dde1